Department of Computer Science and Engineering
Indian Institute of Technology Guwahati

Implementation of Programming Languages Lab: CS348
3rd Year CSE

Assignment - 3: Lexer for nanoC
Assign Date: *March 16, 2026* Submit Date: *23:59, March 30, 2026*

1 Preamble – nanoC

This assignment follows the lexical specification of C language from the International Standard **ISO/IEC 9899:1999 (E)**. To keep the assignment within our required scope, we have chosen a subset of the specification as given below. We shall refer to this language as nanoC and subsequently (in a later assignment) specify its grammar from the Phase Structure Grammar given in the C Standard.

The lexical specification quoted here is written using a precise yet compact notation typically used for writing language specifications. We first outline the notation and then present the Lexical Grammar that we shall work with.

2 Notation

In the syntax notation used here, syntactic categories (non-terminals) are indicated by *italic type*, and literal words and character set members (terminals) by **bold type**. A colon (*:*) following a non-terminal introduces its definition. Alternative definitions are listed on separate lines, except when prefaced by the words "one of". An optional symbol is indicated by the subscript "opt", so that the following indicates an optional expression enclosed in braces.

{ *expression*$_{opt}$ **}**

3 Lexical Grammar of nanoC

1. **Lexical Elements**

   token_name:
   - *keyword*
   - *identifier*
   - *constant*
   - *string-literal*
   - *punctuator*

2. **Keywords**

   keyword: one of

break	**default**	**return**	**void**
case	**float**	**short**	**double**
char	**for**	**signed**	**while**
else	**unsigned**	**long**	**_Bool**
continue	**if**	**static**	
do	**int**		

3. **Identifiers**

   identifier:
   - *identifier-nondigit*
   - *identifier identifier-nondigit*
   - *identifier digit*

   identifier-nondigit: one of

_	**a**	**b**	**c**	**d**	**e**	**f**	**g**	**h**	**i**	**j**	**k**	**l**	**m**
	n	**o**	**p**	**q**	**r**	**s**	**t**	**u**	**v**	**w**	**x**	**y**	**z**
	A	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**
	N	**O**	**P**	**Q**	**R**	**S**	**T**	**U**	**V**	**W**	**X**	**Y**	**Z**

digit: one of

0 1 2 3 4 5 6 7 8 9

4. **Constants**

constant:
- *integer-constant*
- *floating-constant*
- *character-constant*

integer-constant:
- *nonzero-digit*
- *integer-constant digit*

nonzero-digit: one of

1 2 3 4 5 6 7 8 9

fractional-constant:
- *digit-sequence*$_{opt}$ **.** *digit-sequence*
- *digit-sequence* **.**

sign: one of

+ −

digit-sequence:
- *digit*
- *digit-sequence digit*
- **'** *c-char-sequence* **'**

c-char-sequence:
- *c-char*
- *c-char-sequence c-char*

c-char:
- any member of the source character set except
  - the single-quote **'**, backslash ****, or new-line character
- *escape-sequence*

escape-sequence: one of

**\' \" \? **
\a \b \f \n \r \t \v

5. **String literals**

string-literal:
- **"** *s-char-sequence*$_{opt}$ **"**

s-char-sequence:
- *s-char*
- *s-char-sequence s-char*

s-char:
- any member of the source character set except
  - the double-quote **"**, backslash ****, or new-line character
- *escape-sequence*

6. **Punctuators**

punctuator: one of

```
[ ]  ( )  { }  .  ->
++  --  &  *  +  -  ~  !
/  %  <<  >>  <  >  <=  >=  ==  !=  ^  |  &&  ||
?  :  ;  ...
=  *=  /=  %=  +=  -=  <<=  >>=  &=  ^=  |=
,  #
```

7. **Comments**

   (a) *Multi-line Comment*

   Except within a character constant, a string literal, or a comment, the characters /* introduce a comment. The contents of such a comment are examined only to identify multibyte characters and to find the characters */ that terminate it. Thus, /* **...** */ comments do not nest.

   (b) *Single-line Comment*

   Except within a character constant, a string literal, or a comment, the characters // introduce a comment that includes all multibyte characters up to, but not including, the next new-line character. The contents of such a comment are examined only to identify multibyte characters and to find the terminating new-line character.

4 The Assignment

1. Write a flex specification for the language of nanoC using the above lexical grammar. The flex code should also create a symbol table for all the identifiers. The output must be a series of tokens with line numbers written to an output file (a3_*roll_token*.txt), a symbol table consisting of identifier entries (a3_*roll_st*.txt). If there are errors, they must be reported with the appropriate line numbers. Name of your lex file should be a3_*roll*.l. **("roll" must be your respective Roll Number!)**

2. Prepare a Makefile to compile the specifications and generate the lexer.

3. Prepare a test input file a3_*roll*_test.nc that will test all the lexical rules that you have coded.

4. Prepare a zip file with the name a3_*roll*.zip/tar containing all the above files and upload to Moodle.

5 Marks Distribution

1. Flex Specifications: **60%**

2. File Names & Makefile: **20%**

3. Readme & Test file: **20%**

6 Example flex Specification (a3_roll.l)

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

DIGIT       [0-9]*
DN          [1-9]+
ID          [a-zA-Z_][a-zA-Z0-9_]*
OPERATOR    [+\-*/=]
NUM         {DN}{DIGIT}

%%

"if"         { printf("test"); printf("<KEYWORD, if>\n"); }
"else"       { printf("<KEYWORD, else>\n"); }

{ID}         { printf("<IDENTIFIER, %s>\n", yytext); }
{NUM}        { printf("<NUMBER, %s>\n", yytext); }
{OPERATOR}   { printf("<OPERATOR, %s>\n", yytext); }

"("          { printf("<LEFT_PAREN, (>\n"); }
")"          { printf("<RIGHT_PAREN, )>\n"); }
"{"          { printf("<LEFT_BRACE, {>\n"); }
"}"          { printf("<RIGHT_BRACE, }>\n"); }
";"          { printf("<SEMICOLON, ;>\n"); }

[ \t\n]      ;
"//".*       ;

.            { printf("<UNKNOWN, %s>\n", yytext); }

%%

int main() {
    printf("Enter you test sample:\n");
    yylex();
    return 0;
}
```

6.1 Commands:

1. flex *a3_roll.l*
2. gcc lex.yy.c -lfl
3. ./a.out < *a3_roll_test.nc*